

Trinary: A Ternary (Base-3) Computer Simulation in Pure Python

May 25, 2026

Abstract

Trinary is a complete ternary (base-3) computer system simulated entirely in pure Python. The project spans the full abstraction hierarchy — from fundamental logic gates (TNOT, TAND, TOR) and a ripple-carry adder, through a general-purpose CPU with 26 instructions, memory-mapped I/O, interrupts, and a hardware timer, up to a minimal operating system shell and a PyQt6 desktop visual simulator. This document describes the architecture, instruction set, memory model, interrupt system, and implementation decisions behind the simulator, and serves as a reference for further research into non-binary computing models.

Contents

1	Introduction to Ternary Computing	3
1.1	Binary vs. Ternary Number Systems	3
1.2	Information-Theoretic Advantages	3
1.3	The Soviet Setun Computer (1958)	3
1.4	Why Simulate a Ternary Computer in Software?	4
1.5	Scope of This Book	4
1.6	Verifying Your Environment	5
2	Data Representation	5
3	Logic Gates	5
4	Addition Paths	6
5	CPU Architecture	6
5.1	Registers	6
5.2	Memory Map	6
5.3	Fetch-Decode-Execute Cycle	7
5.3.1	Fetch	7
5.3.2	Decode	7
5.3.3	Execute	7
5.3.4	Update	7
5.4	Dual Stacks	7
5.5	Flags	7
5.6	Interrupts	7

6	Instruction Set	8
6.1	Data Movement	8
6.2	Arithmetic	8
6.3	Logical	8
6.4	Comparison	8
6.5	Control Flow	8
6.6	Subroutines	8
6.7	Stack	8
6.8	Memory	8
6.9	Interrupts	8
6.10	Timer	8
6.11	System	8
7	Assembler	9
8	Machine Code	9
9	OS Shell	9
10	Display System	9
11	PyQt6 Desktop UI	9
12	Testing	10
13	Related Work	10
14	Future Directions	11

1 Introduction to Ternary Computing

1.1 Binary vs. Ternary Number Systems

Modern computing is overwhelmingly binary. Since the earliest electronic computers — ENIAC (1945), EDVAC, and the von Neumann architecture — the two-state switch (on/off, true/false, 1/0) has dominated digital logic. Claude Shannon's 1937 master's thesis [1] established that Boolean algebra could be realized with electrical switching circuits, cementing binary as the natural choice for hardware.

Yet binary is not the only possibility. A ternary (base-3) digit — called a *trit* — can represent three distinct states, typically denoted 0, 1, and 2. This extra state per digit has profound implications for information density, circuit complexity, and arithmetic efficiency.

Table 1: Comparing the number of digits needed to represent values in binary vs. ternary.

Value	Binary digits	Ternary digits	Savings
10	4	3	25%
100	7	5	29%
1000	10	7	30%
10^6	20	13	35%
10^8	27	17	37%

1.2 Information-Theoretic Advantages

Donald Knuth, in *The Art of Computer Programming*, notes that the base r that maximizes information density is e , with 3 being the closest integer [2]. The *radix economy* $E(r, N) = r \cdot \lceil \log_r N \rceil$ measures the cost of representing a number N in base r . Base 3 minimizes this cost among integer bases:

- Binary ($r = 2$): $2 \cdot \log_2 N$
- Ternary ($r = 3$): $3 \cdot \log_3 N \approx 3 \cdot \frac{\ln N}{\ln 3} \approx 2.73 \cdot \ln N$
- Quaternary ($r = 4$): $4 \cdot \log_4 N \approx 4 \cdot \frac{\ln N}{\ln 4} \approx 2.89 \cdot \ln N$

Ternary achieves the lowest constant factor (≈ 2.73) among integer bases, meaning it can represent a given range of values with the fewest physical components in an idealized circuit model. Hurst further explores these tradeoffs in the context of multiple-valued logic systems [3].

1.3 The Soviet Setun Computer (1958)

The most famous practical implementation of ternary computing is the Setun computer, developed at Moscow State University by Nikolai P. Brusentsov and his team in 1958 [4].

The Setun used *balanced ternary* (trits valued $-1, 0, +1$), which offered a natural representation of negative numbers without a dedicated sign bit. Key specifications included:

- 36-bit (trit) word length — approximately 18 decimal digits of precision
- Ferrite-diode memory cells for the register file
- Magnetic drum memory for bulk storage

- Approximately 50 machines were produced and deployed in Soviet universities and research laboratories

The Setun demonstrated that ternary computing was not merely a theoretical curiosity but a viable engineering alternative. Its production ended in the mid-1960s as binary hardware — simpler to design with transistor-transistor logic (TTL) — became dominant.

1.4 Why Simulate a Ternary Computer in Software?

Building a physical ternary computer today would require custom logic families (e.g., CNTFET-based ternary gates [5]) and fabrication processes that remain niche. However, a software simulation offers several advantages:

1. **Pedagogy:** A simulated machine can be inspected at every level — from individual trits and logic gates up to running programs and an operating system — making it an ideal teaching tool for computer architecture.
2. **Experimentation:** Instruction sets, memory models, addressing modes, and interrupt systems can be prototyped, modified, and benchmarked without hardware constraints.
3. **Reproducibility:** A pure-Python simulator runs on any platform with a Python interpreter (3.10+), ensuring that results are fully deterministic and shareable.
4. **Accessibility:** No specialized hardware or FPGA toolchains required — a single `pip install` is sufficient to boot the simulated machine and interact with its OS shell.

1.5 Scope of This Book

This book traces the complete stack of the Trinary ternary computer simulation, progressing through 25 chapters from first principles to a working graphical debugger:

- (Ch. 2) System architecture and module dependency graph
- (Ch. 3) Environment setup and “Hello, Ternary”
- (Ch. 4–5) The Trit datatype and base-conversion systems
- (Ch. 6–7) Ternary logic gates and adder circuits
- (Ch. 8–9) Arithmetic via decimal round-trip and the ALU
- (Ch. 10–11) Register file and memory architecture
- (Ch. 12–20) CPU design, all 26 instructions, interrupts, and timer
- (Ch. 21–22) Two-pass assembler and machine-code encoding
- (Ch. 23) Memory-mapped display and keyboard subsystem
- (Ch. 24) The operating system shell
- (Ch. 25) The PyQt6 visual simulator

Each chapter includes runnable code examples that readers can execute immediately against the Trinary package.

1.6 Verifying Your Environment

Before proceeding, verify that your Python environment is ready:

Listing 1: Installing Trinary and running the test suite.

```
1 pip install -e .
2 python -m pytest tests/ -v
```

The test suite (113 tests, all passing) exercises every component from base conversion to interrupt handling. A successful run produces output similar to:

```
1 ===== 113 passed in 0.06s =====
```

You can also launch the interactive OS shell immediately:

```
1 python -m trinary.os
```

If the shell boots and displays a `Trishell!>` prompt, your environment is fully functional and you are ready to begin.

2 Data Representation

All numeric values are represented as signed-magnitude ternary strings. A leading minus sign (-) denotes a negative number; its absence denotes a non-negative number. The `Trit` class encapsulates a single ternary digit with possible values 0, 1, 2.

Table 2: Ternary-decimal correspondence.

Ternary	Decimal
0	0
1	1
2	2
10	3
11	4
12	5
20	6
21	7
22	8
100	9
-10	-3
-101	-10

Conversion functions provide bidirectional translation between ternary strings, decimal integers, and binary integers, enabling seamless interaction with Python's native numeric operations.

3 Logic Gates

Ternary logic extends Boolean logic by introducing a third truth value. Three primitive gates are defined:

$$\begin{aligned}\text{TNOT}(x) &= 2 - x \\ \text{TAND}(x, y) &= \min(x, y) \\ \text{TOR}(x, y) &= \max(x, y)\end{aligned}$$

where $x, y \in \{0, 1, 2\}$. Truth tables for each gate are available through the `ternary.logic` module.

4 Addition Paths

The project provides two parallel addition implementations:

Native adder (`adder.py`) A half-adder, full-adder, and ripple-carry adder operating directly on ternary digits (values 0, 1, 2). The ripple-carry adder chains full-adders for each trit position and does not handle negative numbers.

Decimal round-trip (`arithmetic.py`) The ADD, SUB, MUL, and DIV functions convert ternary operands to decimal, perform the operation in Python integers, and convert the result back to ternary. This path naturally handles signed-magnitude negatives.

The ALU (`alu.py`) selects the latter path for all arithmetic operations.

5 CPU Architecture

5.1 Registers

The CPU provides four general-purpose registers and three special-purpose registers:

Table 3: Register file.

Name	Purpose
R0	Primary accumulator
R1	Secondary register
R2	General purpose
R3	General purpose (OS convention: always 0)
PC	Program counter
SP	Stack pointer (starts at 255, grows downward)
Flags	Condition flags: ZERO, EQUAL, GREATER, LESS

5.2 Memory Map

The address space is 512 trits wide. Memory is divided into regions:

Table 4: Memory map.

Addresses	Purpose
0–127	General data / program
128–199	Stack
200–255	Video RAM (56 characters, 7×8 grid)
260	Keyboard buffer
256–511	Reserved / future expansion

5.3 Fetch-Decode-Execute Cycle

5.3.1 Fetch

The instruction at `program[PC]` is read as a raw string.

5.3.2 Decode

The instruction string is split into tokens. The first token is the opcode; subsequent tokens are operands (register names, numeric literals, or labels).

5.3.3 Execute

The CPU dispatches to the appropriate handler for the decoded opcode. Side effects include register changes, memory writes, stack operations, and PC modification.

5.3.4 Update

Unless the instruction modified PC (e.g., `JMP`, `JZ`, `CALL`), PC is incremented by one.

5.4 Dual Stacks

The CPU maintains two distinct stacks:

Memory stack (SP, addresses 128–199) — used by `PUSH` and `POP`. Grows downward from address 255.

Call stack (Python list) — used by `CALL` and `RET` for subroutine return addresses. Not visible in the memory address space.

This design separates data stacking from control-flow stacking, simplifying implementation and avoiding address-space conflicts.

5.5 Flags

The `CMP` instruction compares two ternary values and sets four boolean flags:

- **ZERO**: result is zero.
- **EQUAL**: operands are equal.
- **LESS**: first operand < second.
- **GREATER**: first operand > second.

Jump instructions (`JZ`, `JNZ`) and conditional logic use these flags.

5.6 Interrupts

The interrupt system provides:

- **Interrupt vector table (IVT)**: 8 entries, each storing a handler address. Set via `SETIVT`.
- **Software interrupts**: triggered by the `INT` instruction with an interrupt number argument.
- **Hardware timer**: periodic interrupt (interrupt 0) configured via `SETTIMER`. Can be enabled/disabled with `EI/DI`.
- **Interrupt return**: `IRET` pops the saved PC and re-enables interrupts.

6 Instruction Set

The CPU implements 26 instructions, grouped into functional categories:

6.1 Data Movement

LOAD, MOV, CLR

6.2 Arithmetic

ADD, SUB, MUL, DIV

6.3 Logical

AND, OR, NOT

6.4 Comparison

CMP

6.5 Control Flow

JMP, JZ, JNZ

6.6 Subroutines

CALL, RET

6.7 Stack

PUSH, POP

6.8 Memory

STOREM, LOADM

6.9 Interrupts

INT, IRET, EI, DI, SETIVT

6.10 Timer

SETTIMER

6.11 System

HALT

See docs/instruction-set.md for the complete opcode reference.

7 Assembler

The assembler (`assembler.py`) is a two-pass assembler:

1. **Pass 1:** Scan all source lines for label definitions (e.g., `start:`). Record the address (line number) of each label in a symbol table.
2. **Pass 2:** Re-scan all source lines, resolve label references in jump/call operands using the symbol table, and emit a list of instruction strings.

Inline comments are supported with both `#` and `;` delimiters, handled by `assembler.parse_line`.

8 Machine Code

The `machine.py` module provides a machine-code encoder and decoder that converts between assembly strings and variable-length ternary opcode strings. Currently, 21 of the 26 instructions have machine-code mappings. The instructions `INT`, `IRET`, `EI`, `DI`, `SETIVT`, and `SETTIMER` do not yet have opcode assignments in the encoding table.

9 OS Shell

The minimal operating system (`os.py`) is written entirely in assembly (342 lines). It provides an interactive command shell with the following commands:

Table 5: OS shell commands.

Command	Description
HELP	Lists available commands
CLS	Clears the display
REGS	Shows register values
MEMDUMP	Dumps first 8 memory addresses
RUN	Placeholder: prints "RUN!"

The OS implements memory-mapped keyboard polling, an input line buffer with backspace support, cursor management with display scrolling, and multi-character command parsing.

10 Display System

The display system maps memory addresses 200–255 to a 7-row by 8-column character display (56 characters total). A `PixelDisplay` class provides a 27×27 pixel framebuffer representation. Address 260 serves as a keyboard buffer: the OS polls this address, and the UI (or a human user) writes ASCII key codes into it.

11 PyQt6 Desktop UI

The `trinary.ui` package provides a PyQt6-based graphical simulator with the following panels:

- **Assembler editor:** syntax highlighting (opcodes=cyan, registers=yellow, labels=purple, comments=green, numbers=white), clickable breakpoints in the gutter.

- **Machine code viewer:** ternary opcode strings displayed alongside assembly source.
- **Register panel:** R0–R3 values with flash animation on change; PC, SP, and four flag indicators.
- **Memory viewer:** 512-row table with colour-coded access tracking (green=write, blue=read, current PC highlight).
- **Stack viewer:** live stack contents with SP position marker.
- **Execution trace:** step-by-step history (step number, PC, instruction, registers, flags).
- **Pixel display:** 27×27 pixel framebuffer with keyboard capture.
- **Toolbar:** demo selector (13 programs) and execution controls (Assemble, Run, Step Into, Step Over, Reset, Pause, Continue).

The UI is fully decoupled from the back-end – it interacts exclusively through the public CPU and memory APIs. The stylesheet (`styles.py`) provides a dark cyberpunk theme.

12 Testing

The project includes 113 tests across 8 test files:

Table 6: Test suite.

File	Coverage
<code>test_conversion.py</code>	Trit class, base conversion
<code>test_arithmetic.py</code>	ADD, SUB, MUL, DIV
<code>test_alu.py</code>	All 8 ALU operations
<code>test_assembler.py</code>	Label resolution, comments
<code>test_cpu.py</code>	All 26 opcodes, flags, stack, interrupts
<code>test_cpu_stress.py</code>	Nested calls, long loops
<code>test_display.py</code>	Memory-mapped display, keyboard

Tests can be run with:

```
1 python -m pytest tests/ -v
```

13 Related Work

The most famous ternary computer is the Soviet Setun (1958) [4], which used balanced ternary (−1, 0, +1) and was built from ferrite-diode cells. More recently, ternary logic has been explored in the context of CNTFET-based circuits [5] and memristive ternary systems.

Software simulators of non-binary architectures include the ternary-computer project and various university-level teaching simulators. Trinary distinguishes itself by combining a complete, documented ISA with an interactive OS shell and a modern desktop UI, all implemented in a single, well-tested Python package.

14 Future Directions

Possible extensions and improvements:

- Complete machine-code mappings for the remaining six instructions (INT, IRET, EI, DI, SETIVT, SETTIMER).
- A balanced ternary mode ($-1, 0, +1$) as an alternative to signed-magnitude representation.
- An expanded memory model supporting paging or banking.
- A network interface or I/O port system.
- A compiler from a high-level language (e.g., a subset of C) to ternary assembly.
- WebAssembly compilation via Pyodide for in-browser simulation.

References

- [1] Claude E. Shannon. "A Symbolic Analysis of Relay and Switching Circuits". MA thesis. Massachusetts Institute of Technology, 1937.
- [2] Donald E. Knuth. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3rd. Balanced ternary is discussed in Section 4.1. Addison-Wesley, 2003. Chap. 4.1 – Positional Number Systems.
- [3] S. L. Hurst. *Multiple-Valued Logic: Its Status and Its Future*. Los Alamitos, CA: IEEE Computer Society Press, 1990.
- [4] N. P. Brusentsov. "The Setun computer". In: *Vestnik Moskovskogo Universiteta, Ser. Matematika, Mekhanika* 4 (1965), pp. 61–68.
- [5] S. Lin, Y. Zhang, et al. "Ternary logic circuits based on CNTFETs: a review". In: *IEEE Transactions on Circuits and Systems* 69.3 (2022), pp. 1000–1013.